

# Estructura del Proyecto CSP Parser

Proyecto CSP Parser

2026

## Contents

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>Estructura del Proyecto CSP Parser &amp; Evaluator</b> | <b>2</b> |
| Organización Modular y Sistema de Compilación . . . . .   | 2        |
| 1. Visión General del Proyecto . . . . .                  | 2        |
| 1.1 Características Principales . . . . .                 | 2        |
| 2. Estructura de Directorios . . . . .                    | 3        |
| 3. Módulos del Sistema . . . . .                          | 4        |
| 3.1 Bibliotecas Base (src/) . . . . .                     | 4        |
| 3.2 Programas del Pipeline (src/) . . . . .               | 5        |
| 3.3 Bibliotecas C (src/) . . . . .                        | 7        |
| 4. Sistema de Compilación . . . . .                       | 8        |
| 4.1 Makefile Modular . . . . .                            | 8        |
| 4.2 Targets Disponibles . . . . .                         | 8        |
| 4.3 Proceso de Compilación . . . . .                      | 8        |
| 4.4 Flags de Compilación . . . . .                        | 8        |
| 4.5 Dependencias por Programa . . . . .                   | 9        |
| 5. Pipeline de Ejecución . . . . .                        | 9        |
| 5.1 Flujo de Datos . . . . .                              | 9        |
| 5.2 Script de Pipeline . . . . .                          | 10       |
| 6. Tipos de Variables . . . . .                           | 11       |
| 6.1 Tipos Soportados . . . . .                            | 11       |
| 6.2 Grid Bitmask (vkNumeric) . . . . .                    | 11       |
| 7. AST (Abstract Syntax Tree) . . . . .                   | 11       |
| 7.1 Tipos de Nodos (TASTNodeKind) . . . . .               | 11       |
| 7.2 Estructura de Nodo . . . . .                          | 12       |
| 8. Funciones . . . . .                                    | 12       |
| 8.1 Funciones Builtin . . . . .                           | 12       |
| 8.2 Funciones User-Defined . . . . .                      | 12       |
| 9. Archivos de Prueba . . . . .                           | 13       |
| 9.1 Tests Unitarios (test/) . . . . .                     | 13       |
| 9.2 Ejemplos JSON (json/) . . . . .                       | 13       |
| 10. Documentación . . . . .                               | 13       |
| 10.1 Archivos Markdown (docs/) . . . . .                  | 13       |
| 10.2 Referencias Rápidas (docs/*.txt) . . . . .           | 14       |
| 10.3 PDFs Técnicos (docs/*.pdf) . . . . .                 | 14       |
| 11. Métricas del Proyecto . . . . .                       | 14       |

|                                                      |    |
|------------------------------------------------------|----|
| 11.1 Código Fuente . . . . .                         | 14 |
| 11.2 Compilación . . . . .                           | 14 |
| 11.3 Ejecutables . . . . .                           | 15 |
| 11.4 Documentación . . . . .                         | 15 |
| 12. Ventajas de la Arquitectura Modular . . . . .    | 15 |
| 12.1 Filosofía Unix . . . . .                        | 15 |
| 12.2 Beneficios . . . . .                            | 15 |
| 13. Flujo de Trabajo . . . . .                       | 16 |
| 13.1 Compilar el Proyecto . . . . .                  | 16 |
| 13.2 Ejecutar Pipeline Completo . . . . .            | 16 |
| 13.3 Ejecutar Etapas Individualmente . . . . .       | 16 |
| 13.4 Ejecutar Tests Unitarios . . . . .              | 16 |
| 13.5 Agregar Nueva Función User-Defined . . . . .    | 17 |
| 14. Integración con GecodeInfo . . . . .             | 17 |
| 14.1 Análisis del Espacio de Soluciones . . . . .    | 17 |
| 14.2 Bug Crítico Resuelto (Interval Nodes) . . . . . | 18 |
| 15. Próximos Pasos . . . . .                         | 18 |
| 15.1 Funcionalidades Pendientes . . . . .            | 18 |
| 15.2 Mejoras de Infraestructura . . . . .            | 18 |
| 15.3 Documentación . . . . .                         | 18 |
| 16. Conclusiones . . . . .                           | 19 |
| 16.1 Logros . . . . .                                | 19 |
| 16.2 Estado del Proyecto . . . . .                   | 19 |
| 16.3 Filosofía de Diseño . . . . .                   | 19 |

## Estructura del Proyecto CSP Parser & Evaluator

### Organización Modular y Sistema de Compilación

**Fecha:** 21 de febrero de 2026 **Versión:** 2.0 - Arquitectura Modular Unix

---

## 1. Visión General del Proyecto

CSP Parser & Evaluator es un pipeline de verificación, evaluación y propagación de restricciones para problemas de Constraint Satisfaction (CSP), diseñado con filosofía Unix: programas pequeños, cada uno hace una cosa, se pueden encadenar.

### 1.1 Características Principales

- Validación semántica con 8 reglas
- Construcción de hipergrafos CSP desde JSON
- Verificación de funciones user-defined (.o/.so)
- Propagación de dominios (AC-3, forward/backward consistency)
- 4 tipos de variables: boolean, set, integer, numeric
- Aritmética de intervalos + grid bitmask (20 puntos)
- 9 programas standalone componibles
- Salida JSON en cada etapa

---

## 2. Estructura de Directorios

```
/home/rodo/parser/
|- src/                                # Código fuente (49 archivos)
|   |- MiniSys.pas                     # Sistema base (strings, floats, file I/O)
|   |- MiniJSON.pas                    # Parser/writer JSON mínimo
|   |- MiniMath.pas                    # Interfaz a C math library
|   |- ExpressionAST.pas                # Nodos AST (33 tipos)
|   |- PrattParser.pas                  # Pratt parser (precedencia)
|   |- LabeledSet.pas                   # Dominios discretos (sets)
|   |- ASTEvaluator.pas                 # Evaluador de expresiones
|   |- SyntaxChecker.pas                # Validación semántica (8 reglas)
|   |- JsonToGraph.pas                  # Sistema JSON -> hipergrafo CSP
|   |- FunctionChecker.pas              # Verificación .o/.so
|   |- CSPEval.pas                      # Propagador AC-3 + grid bitmask
|   |- ForwardChain.pas                 # Forward chaining con trazado
|   |- FwdConsistency.pas               # AC-3 hacia adelante
|   |- BwdConsistency.pas               # Proyección inversa
|   |- JsonSource.pas                   # SQLite -> JSON
|   |- JsonSink.pas                     # JSON -> SQLite
|   |- minimath_trig.c                  # sin, cos, tan
|   |- minimath_exp.c                   # exp, ln, log
|   |- minimath_util.c                  # abs, sqrt, sqr
|   |- minimath_interval.c              # IvMake, IvAdd, IvMul, IvDiv, etc.
|   |- minimath_complex.c               # Aritmética compleja
|   |- minimath_grid.c                  # mg_narrow_gt/ge/lt/le/iv (bitmask)
|   |- minimath.h                       # Header compartido
|   |- roaring.c                        # Roaring bitmaps (conjuntos)
|
|- obj/                                # Archivos objeto (1.2 MB)
|   |- *.o                             # Código objeto Pascal
|   |- *.ppu                           # Pascal unit files
|   |- roaring.o                        # Roaring bitmap (C)
|   |- minimath_*.o                    # Bibliotecas matemáticas (C)
|   |- minimath_grid.o                 # Grid bitmask filter (C)
|   |- libsqlite3.so                   # Symlink a SQLite3
|
|- docs/                               # Documentación
|   |- README.md                       # Índice de documentación
|   |- estructura_proyecto.md           # Este documento
|   |- 01_introduccion.md              # Manual completo
|   |- 02_arquitectura_pipeline.md     # Pipeline CSP
|   |- *.txt                           # Referencias rápidas (1 por programa)
|   |- *.pdf                           # PDFs técnicos (2.3 MB)
|
|- json/                               # Ejemplos JSON
```

```

|      |- *.json          # Casos de uso completos
|
|- test/                  # Tests compilados
|   |- TestRoaring        # Test de Roaring bitmaps
|   |- TestLabeledSet     # Test de dominios discretos
|   |- TestMiniMath       # Test de funciones matemáticas
|   |- TestMiniComplex    # Test de aritmética compleja
|   |- TestMiniInterval   # Test de aritmética de intervalos
|   |- TestMiniFormat     # Test de formateo
|   |- TestMiniUtils      # Test de utilidades
|
|- BwdConsistency         # Ejecutable (272 KB)
|- CSPEval                # Ejecutable (264 KB)
|- ForwardChain           # Ejecutable (268 KB)
|- FunctionChecker        # Ejecutable (221 KB)
|- FwdConsistency         # Ejecutable (272 KB)
|- JsonSink               # Ejecutable (225 KB)
|- JsonSource             # Ejecutable (210 KB)
|- JsonToGraph            # Ejecutable (256 KB)
|- SyntaxChecker          # Ejecutable (261 KB)
|
|- Makefile               # Sistema de compilación
|- README.md              # Documentación principal
|- csp-pipeline.sh        # Script de pipeline completo

```

---

### 3. Módulos del Sistema

#### 3.1 Bibliotecas Base (src/)

**MiniSys.pas** - Sistema base sin dependencias RTL - UpperCase, LowerCase, Trim - IntToStr, StrToInt, StrToIntDef - FloatToStr, FloatToStrF, TryStrToFloat, StrToFloat - Format, StringOfChar - FileExists

**MiniJSON.pas** - Parser/writer JSON mínimo - Jerarquía TJSONValue: TJSONObject, TJSONArray, TJSONString, TJSONNumber, TJSONBoolean, TJSONNull - ParseJSON(text) -> TJSONValue - WriteJSONCompact(root) -> string - Sin dependencias externas

**MiniMath.pas** - Interfaz Pascal a bibliotecas C - Declaraciones externas a minimath\_\*.c - sin, cos, tan, asin, acos, atan - exp, ln, log, pow, sqrt, abs, sqr - **Interval arithmetic**: IvMake, IvAdd, IvSub, IvMul, IvDiv, IvScale, IvContains, IvIsEmpty, IvIntersect, IvHull - **Grid bitmask**: mg\_narrow\_gt, mg\_narrow\_ge, mg\_narrow\_lt, mg\_narrow\_le, mg\_narrow\_iv

**ExpressionAST.pas** - Nodos AST (33 tipos) - TASTNode: id, kind: TASTNodeKind, left, right, value - TASTNodeKind: CT\_ADD, CT\_SUB, CT\_MUL, CT\_DIV, CT\_AND, CT\_OR, CT\_NOT, CT\_EQ, CT\_LT, CT\_GT, CT\_IN, CT\_UNION, CT\_INTERSECT, CT\_CALL, CT\_INTERVAL, ... - TFuncEntry: name, inputs, outputs, builtin: boolean - IsBuiltinFunction(name) -> boolean - **Builtin functions**: abs, sqrt, sin, cos, tan, ln, exp, log, sqr, interval, complex, ...

**PrattParser.pas** - Pratt parser (precedencia de operadores) - ParseExpression(tokens) ->

**TASTNode** - Precedencia: OR < AND < IMPLICA < =/<>/</> < +/- < \*// < NOT/- (unary) < () (call) - Distingue f (variable) vs f() (función) - Soporta {A, B, C} como sets literales - Soporta interval(lo, hi) como numeric literal

**LabeledSet.pas** - Dominios discretos (sets) - TLabeledSet: Labels: array of string, Active: TBoolArray - AddLabel(s), IsActive(idx), SetActive(idx, val) - Count, IsEmpty, Intersect, Union, Difference

**ASTEvaluator.pas** - Evaluador de expresiones - Evaluate(node, vars) -> TJSONValue - Soporta todas las operaciones AST - Propagación de incertidumbre (variables con múltiples valores)

### 3.2 Programas del Pipeline (src/)

**SyntaxChecker.pas** - Validación semántica (8 reglas)

| Regla | Descripción                   | Salida JSON                              |
|-------|-------------------------------|------------------------------------------|
| 1     | Variables declaradas sin usar | error                                    |
| 2     | Funciones declaradas sin usar | error                                    |
| 3     | Variables usadas sin declarar | error                                    |
| 4     | Funciones usadas sin declarar | error si no es builtin, OK si es builtin |
| 5     | Redeclaración de variable     | error                                    |
| 6     | Redeclaración de función      | error                                    |
| 7     | Tipo incorrecto en expresión  | error                                    |
| 8     | Override de builtin           | error                                    |

**Input:** system.json -> **Output:** JSON con {"status": "OK"} o {"errors": [...]}

**JsonToGraph.pas** - Sistema JSON -> hipergrafo CSP

**Input:** system.json (validado por SyntaxChecker) **Output:** graph.json con:

```
{
  "nodes": [{"id": 0, "kind": "CT_ADD", "left": 1, "right": 2, ...}],
  "vars": [{"id": 0, "name": "x", "kind": "vkNumeric", "dom_lo": 0.0, "dom_hi": 100.0, ...}],
  "constrs": [{"id": 0, "tree_id": 5, "var_refs": [0,1], "func_refs": []}],
  "funcs": [{"id": 0, "name": "mifunc", "inputs": [...], "outputs": [...]}]
}
```

**Transformaciones:** - Variables -> vars array con id, name, kind (vkBoolean, vkSet, vkInteger, vkNumeric) - Expresiones -> AST nodes + constraints con var\_refs (IDs de variables usadas) - Funciones user-defined -> funcs array con func\_refs en constraints - **IMPORTANTE:** Builtins NO aparecen en func\_refs (motor las resuelve internamente)

**FunctionChecker.pas** - Verificación .o/.so de funciones

**Input:** graph.json (salida de JsonToGraph) **Output:** Exit code 0 (OK) / 1 (missing) / 2 (fatal error)

**Search path:** default = . ./lib ./obj /usr/local/lib/csp **Override:** --path dir1:dir2:...

Para cada función en `graph.json["funcs"]`: 1. Buscar {funcname}.so en search path 2. Si no existe, buscar {funcname}.o 3. Si ninguno existe -> error

### Ejemplo:

```
./FunctionChecker graph.json --path ./lib:./obj:/usr/local/lib/csp
```

---

**CSPEval.pas** - Propagador AC-3 con grid bitmask

**Input:** graph.json (después de FunctionChecker OK) **Output:** result.json con dominios propagados

**Algoritmo:** 1. Cargar grafo: vars, nodes, constraints 2. Para variables numeric: inicializar grid 20-pt + bitmask 3. Iterar hasta punto fijo: - Para cada constraint: - Evaluar expresión con dominios actuales - Narrowing con grid bitmask (`mg_narrow_*`) - Actualizar dominios de variables 4. Output: {"status": "CONSISTENT"|"FAILED", "iterations": N, "vars": [...]}

**Grid bitmask (vkNumeric):** - GridPts: array[0..20] of Double (21 puntos) - GridMask: LongWord (32 bits, usa 21) - `mg_narrow_gt/ge/lt/le/iv(pts, mask, n, thresh)` -> nueva mask  
- Después de narrowing: actualizar Active[] y recalcular bounds

---

**ForwardChain.pas** - Forward chaining con trazado

**Input:** graph.json **Output:** JSON con trazado paso a paso

```
{
  "steps": [
    {"step": 0, "constraint": 0, "action": "evaluate", "result": {...}},
    {"step": 1, "constraint": 1, "action": "narrow", "var": 0, "old_domain": [...], "new_domain": [...]},
  ],
  "final_state": {...}
}
```

Útil para debugging y visualización del proceso de propagación.

---

**FwdConsistency.pas** - AC-3 hacia adelante

Similar a CSPEval pero verifica consistencia sin modificar dominios. **Output:** {"consistent": true|false}

---

**BwdConsistency.pas** - Proyección inversa

Propaga restricciones hacia atrás por el árbol de expresiones. **Input:** graph.json **Output:** JSON con dominios refinados

---

### JsonSource.pas - SQLite -> JSON

Lee tabla SQLite y genera JSON para pipeline. **Input:** SQLite DB path + query **Output:** JSON en stdout

```
./JsonSource database.db "SELECT * FROM variables" > input.json
```

---

### JsonSink.pas - JSON -> SQLite

Lee JSON de stdin y guarda en tabla SQLite. **Input:** JSON en stdin **Output:** SQLite DB updated

```
cat result.json | ./JsonSink output.db results_table
```

## 3.3 Bibliotecas C (src/)

**minimath\_trig.c** - Funciones trigonométricas - mm\_sin, mm\_cos, mm\_tan - mm\_asin, mm\_acos, mm\_atan, mm\_atan2 - Compilado: gcc -c -O2 -std=c99 minimath\_trig.c -o obj/minimath\_trig.o

**minimath\_exp.c** - Funciones exponenciales - mm\_exp, mm\_ln, mm\_log, mm\_pow - Compilado: gcc -c -O2 -std=c99 minimath\_exp.c -o obj/minimath\_exp.o

**minimath\_util.c** - Utilidades matemáticas - mm\_abs, mm\_sqrt, mm\_sqr - mm\_min, mm\_max, mm\_clamp - Compilado: gcc -c -O2 -std=c99 minimath\_util.c -o obj/minimath\_util.o

**minimath\_interval.c** - Aritmética de intervalos - **Tipos:** struct Interval { double lo, hi; } - **Constructores:** IvMake(lo, hi), IvPoint(x), IvEmpty() - **Operaciones:** IvAdd, IvSub, IvMul, IvDiv, IvScale(iv, k) - **Predicados:** IvContains(iv, x), IvIsEmpty(iv), IvOverlaps(a, b) - **Combinadores:** IvIntersect(a, b), IvHull(a, b) (unión convexa) - Compilado: gcc -c -O2 -std=c99 minimath\_interval.c -o obj/minimath\_interval.o

**minimath\_complex.c** - Aritmética compleja - **Tipos:** struct Complex { double re, im; } - **Operaciones:** CxAdd, CxSub, CxMul, CxDiv - **Funciones:** CxAbs, CxArg, CxConj - Compilado: gcc -c -O2 -std=c99 minimath\_complex.c -o obj/minimath\_complex.o

**minimath\_grid.c** - Grid bitmask filter - **Función:** Narrowing de grid 20-pt con bitmask de 32 bits - **API:** - mg\_narrow\_gt(pts, mask, n, thresh) -> nueva mask (> thresh) - mg\_narrow\_ge(pts, mask, n, thresh) -> nueva mask (>= thresh) - mg\_narrow\_lt(pts, mask, n, thresh) -> nueva mask (< thresh) - mg\_narrow\_le(pts, mask, n, thresh) -> nueva mask (<= thresh) - mg\_narrow\_iv(pts, mask, n, lo, hi) -> nueva mask (dentro de [lo, hi]) - **Uso:** CSPEval, ForwardChain, FwdConsistency, BwdConsistency - Compilado: gcc -c -O2 -std=c99 minimath\_grid.c -o obj/minimath\_grid.o

**roaring.c** - Roaring bitmaps (conjuntos comprimidos) - Implementación de Roaring bitmaps para representación eficiente de conjuntos de enteros - Compilado: gcc -c -O2 -DNDEBUG -D\_FORTIFY\_SOURCE=0 -fno-builtin -mpopcnt roaring.c -o obj/roaring.o

**minimath.h** - Header compartido - Declaraciones de todas las funciones C - Estructuras Interval, Complex - Constantes matemáticas

---

## 4. Sistema de Compilación

### 4.1 Makefile Modular

El Makefile está organizado con targets para cada programa y sus dependencias.

### 4.2 Targets Disponibles

| Target               | Descripción                 | Binario         |
|----------------------|-----------------------------|-----------------|
| make all             | Compila todos los programas | 9 ejecutables   |
| make syntax-checker  | Compilar SyntaxChecker      | SyntaxChecker   |
| make json-graph      | Compilar JsonToGraph        | JsonToGraph     |
| make func-checker    | Compilar FunctionChecker    | FunctionChecker |
| make csp-eval        | Compilar CSPEval            | CSPEval         |
| make forward-chain   | Compilar ForwardChain       | ForwardChain    |
| make fwd-consistency | Compilar FwdConsistency     | FwdConsistency  |
| make bwd-consistency | Compilar BwdConsistency     | BwdConsistency  |
| make json-source     | Compilar JsonSource         | JsonSource      |
| make json-sink       | Compilar JsonSink           | JsonSink        |
| make clean           | Limpiar archivos compilados | -               |

### 4.3 Proceso de Compilación

#### Fase 1: Compilar bibliotecas C

```
# Roaring bitmaps
gcc -c -O2 -DNDEBUG -D_FORTIFY_SOURCE=0 -fno-builtin -mpopcnt \
    src/roaring.c -o obj/roaring.o

# MiniMath modules
gcc -c -O2 -std=c99 src/minimath_trig.c -o obj/minimath_trig.o
gcc -c -O2 -std=c99 src/minimath_exp.c -o obj/minimath_exp.o
gcc -c -O2 -std=c99 src/minimath_util.c -o obj/minimath_util.o
gcc -c -O2 -std=c99 src/minimath_interval.c -o obj/minimath_interval.o
gcc -c -O2 -std=c99 src/minimath_complex.c -o obj/minimath_complex.o
gcc -c -O2 -std=c99 src/minimath_grid.c -o obj/minimath_grid.o
```

#### Fase 2: Compilar programas Pascal

```
# Ejemplo: CSPEval
fpc -Mobjfpc -Sh -O2 -Fusrc -FUobj \
    -k"obj/minimath_trig.o" -k"obj/minimath_exp.o" \
    -k"obj/minimath_util.o" -k"obj/minimath_interval.o" \
    -k"obj/minimath_grid.o" \
    src/CSPEval.pas -o./CSPEval
```

### 4.4 Flags de Compilación

Free Pascal Compiler (FPC):

FPC = fpc

FLAGS = -Mobjfpc -Sh -O2 -Fusrc -FUobj

- -Mobjfpc: Modo ObjFPC (Object Pascal)
- -Sh: Usar ansistrings
- -O2: Optimización nivel 2
- -Fusrc: Unit search path
- -FUobj: Unit output path

## GCC (C):

CC = gcc

CFLAGS = -O2 -std=c99

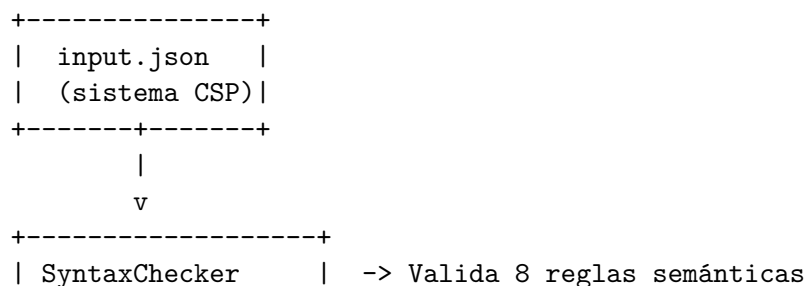
- -O2: Optimización nivel 2
- -std=c99: Estándar C99

## 4.5 Dependencias por Programa

| Programa        | Pascal Units                                            | C Objects                        |
|-----------------|---------------------------------------------------------|----------------------------------|
| SyntaxChecker   | MiniSys, MiniJSON, ExpressionAST, PrattParser           | -                                |
| JsonToGraph     | MiniSys, MiniJSON, ExpressionAST, PrattParser           | -                                |
| FunctionChecker | MiniSys, MiniJSON                                       | -                                |
| CSPEval         | MiniSys, MiniJSON, ExpressionAST, PrattParser, MiniMath | minimath_*.o,<br>minimath_grid.o |
| ForwardChain    | MiniSys, MiniJSON, ExpressionAST, PrattParser, MiniMath | minimath_*.o,<br>minimath_grid.o |
| FwdConsistency  | MiniSys, MiniJSON, ExpressionAST, PrattParser, MiniMath | minimath_*.o,<br>minimath_grid.o |
| BwdConsistency  | MiniSys, MiniJSON, ExpressionAST, PrattParser, MiniMath | minimath_*.o,<br>minimath_grid.o |
| JsonSource      | MiniSys                                                 | libsqlite3.so                    |
| JsonSink        | MiniSys, MiniJSON                                       | libsqlite3.so                    |

## 5. Pipeline de Ejecución

### 5.1 Flujo de Datos



```

+-----+-----+
| (exit 0 = OK)
v
+-----+-----+
| JsonToGraph      | -> Genera hipergrafo CSP
+-----+-----+
| graph.json
v
+-----+-----+
| FunctionChecker  | -> Verifica .o/.so de funciones
+-----+-----+
| (exit 0 = OK)
v
+-----+-----+
| CSPEval          | -> Propaga dominios a punto fijo
| o                |
| ForwardChain     | -> Forward chaining con trazado
+-----+-----+
| result.json
v
+-----+-----+
| output.json      |
| (dominios        |
| propagados)      |
+-----+-----+

```

## 5.2 Script de Pipeline

**csp-pipeline.sh:**

```

#!/bin/bash
# Pipeline CSP completo

INPUT="$1"

if [ -z "$INPUT" ]; then
    echo "Uso: $0 input.json"
    exit 1
fi

# 1. Validar sintaxis
echo "==> SyntaxChecker"
./SyntaxChecker "$INPUT" || exit 1

# 2. Generar grafo
echo "==> JsonToGraph"
./JsonToGraph "$INPUT" > /tmp/graph.json || exit 2

# 3. Verificar funciones

```

```

echo "==> FunctionChecker"
./FunctionChecker /tmp/graph.json --path ./lib:./obj || exit 3

# 4. Propagar dominios
echo "==> CSPEval"
./CSPEval /tmp/graph.json || exit 4

echo "==> Pipeline OK"

```

---

## 6. Tipos de Variables

### 6.1 Tipos Soportados

| Tipo    | Kind      | Dominio                         | Valor                         | Storage         |
|---------|-----------|---------------------------------|-------------------------------|-----------------|
| boolean | vkBoolean | [true, false]                   | subconjunto                   | TLabeledSet     |
| set     | vkSet     | ["A", "B", "C", ...]            | subconjunto                   | TLabeledSet     |
| integer | vkInteger | lista explícita<br>[1,5,10,...] | subconjunto                   | TLabeledSet     |
| numeric | vkNumeric | [lo, hi]                        | [lo', hi'] subset<br>[lo, hi] | Interval + Grid |

---

### 6.2 Grid Bitmask (vkNumeric)

Variables numeric usan una representación híbrida:

**Nivel 1: Interval** - Lo, Hi: bounds del dominio

**Nivel 2: Grid 20-pt** - GridPts: array[0..20] of Double (21 puntos) - GridStep = (Hi - Lo) / 20 - GridPts[k] = Lo + k \* GridStep

**Nivel 3: Bitmask** - GridMask: LongWord (32 bits, usa 21) - Bit k = 1 -> GridPts[k] está activo - Bit k = 0 -> GridPts[k] eliminado por narrowing

**Operaciones:** 1. **Inicialización:** GridMask = 0xFFFFFFFF (todos activos) 2. **Narrowing:** GridMask := mg\_narrow\_gt(GridPts, GridMask, 21, threshold) 3. **Sync:** Actualizar Active[] basado en bits activos 4. **Bounds:** Recalcular Lo, Hi desde puntos activos

**Ventajas:** - Discretización eficiente de dominios continuos - Narrowing rápido con operaciones bitwise - Precisión controlada (20 puntos)

---

## 7. AST (Abstract Syntax Tree)

### 7.1 Tipos de Nodos (TASTNodeKind)

**Aritméticos:** - CT\_ADD, CT\_SUB, CT\_MUL, CT\_DIV

**Relacionales:** - CT\_EQ, CT\_NE, CT\_LT, CT\_GT, CT\_LE, CT\_GE

**Lógicos:** - CT\_AND, CT\_OR, CT\_NOT, CT\_IMPLICA

**Conjuntos:** - CT\_IN, CT\_UNION, CT\_INTERSECT, CT\_DIFFERENCE, CT\_SUBSET, CT\_CARDINALITY

**Literales:** - CT\_INTEGER, CT\_FLOAT, CT\_BOOLEAN, CT\_LABEL, CT\_SET\_LITERAL - CT\_INTERVAL (numeric literal)

**Variables:** - CT\_VAR (referencia a variable por ID)

**Funciones:** - CT\_CALL (llamada a función builtin o user-defined)

## 7.2 Estructura de Nodo

type

```
TASTNode = record
  id: Integer;
  kind: TASTNodeKind;
  left: Integer;  // -1 si no hay hijo izquierdo
  right: Integer; // -1 si no hay hijo derecho
  value: string;  // para literales, identificadores
end;
```

Ejemplo:  $(x + 5) * y$

```
[0] CT_MUL
    left=1, right=2
    [1] CT_ADD
        left=3, right=4
        [3] CT_VAR (x)
        [4] CT_INTEGER (5)
    [2] CT_VAR (y)
```

---

## 8. Funciones

### 8.1 Funciones Builtin

Registradas en ExpressionAST.pas:

```
function IsBuiltinFunction(const Name: string): Boolean;
const
  BuiltinNames: array[0..15] of string = (
    'abs', 'sqrt', 'sqr',
    'sin', 'cos', 'tan', 'asin', 'acos', 'atan',
    'exp', 'ln', 'log', 'pow',
    'interval', 'complex', 'cardinality'
  );
```

**Características:** - NO se declaran en JSON "functions" - Auto-resueltas por SyntaxChecker (Regla 4) - NO aparecen en func\_refs del grafo - Motor las resuelve internamente

### 8.2 Funciones User-Defined

**Declaración en JSON:**

```
{
  "name": "mifunc",
  "inputs": [
    {"name": "x", "type": "numeric", "domain": [0.0, 100.0]}
  ],
  "outputs": [
    {"name": "r", "type": "numeric", "domain": [0.0, 10.0]}
  ]
}
```

**Verificación:** - FunctionChecker busca mifunc.so o mifunc.o en search path - Si no existe -> error - Si existe -> agrega a func\_refs del constraint

**Runtime:** - Motor carga .so con dlopen - Llama función vía dlsym - Signature: void mifunc(double \*inputs, int n\_in, double \*outputs, int n\_out)

## 9. Archivos de Prueba

### 9.1 Tests Unitarios (test/)

| Programa         | Descripción              | Tamaño |
|------------------|--------------------------|--------|
| TestRoaring      | Roaring bitmaps          | 193 KB |
| TestLabeledSet   | Dominios discretos       | -      |
| TestMiniMath     | Funciones matemáticas    | 222 KB |
| TestMiniComplex  | Aritmética compleja      | 226 KB |
| TestMiniInterval | Aritmética de intervalos | 226 KB |
| TestMiniFormat   | Formateo de strings      | 209 KB |
| TestMiniUtils    | Utilidades del sistema   | 214 KB |
| TestFloatConv    | Conversiones float       | 206 KB |
| testex2          | Ejemplo adicional        | 193 KB |

### 9.2 Ejemplos JSON (json/)

Ubicación: /home/rodo/parser/json/

Casos de uso completos demostrando: - Variables de 4 tipos - Operaciones aritméticas, lógicas, relacionales - Operaciones de conjuntos - Funciones builtin - Funciones user-defined (con .o en lib/)

## 10. Documentación

### 10.1 Archivos Markdown (docs/)

| Archivo                | Descripción             | Estado |
|------------------------|-------------------------|--------|
| README.md              | Índice de documentación | [OK]   |
| estructura_proyecto.md | Este documento          | [OK]   |

| Archivo                     | Descripción            | Estado          |
|-----------------------------|------------------------|-----------------|
| 01_introduccion.md          | Manual completo        | [WIP] Pendiente |
| 02_arquitectura_pipeline.md | Pipeline CSP           | [WIP] Pendiente |
| 03_formato_json.md          | Formato JSON           | [WIP] Pendiente |
| 04_motores_propagacion.md   | Motores de propagación | [WIP] Pendiente |

## 10.2 Referencias Rápidas (docs/\*.txt)

Cada programa tiene un archivo `.txt` de referencia rápida: - SyntaxChecker.txt - JsonToGraph.txt - FunctionChecker.txt - CSPEval.txt - ForwardChain.txt - FwdConsistency.txt - BwdConsistency.txt - JsonSource.txt - JsonSink.txt

## 10.3 PDFs Técnicos (docs/\*.pdf)

| Archivo                        | Tamaño | Descripción        |
|--------------------------------|--------|--------------------|
| documento.pdf                  | 116 KB | Documento general  |
| funciones_builtin.pdf          | 5 KB   | Lista de builtins  |
| JsonToGraph-Documentacion.pdf  | 11 KB  | Doc JsonToGraph    |
| nodos_y_funciones.pdf          | 9 KB   | Tipos AST          |
| GeCode_Space_Reglas.pdf        | 129 KB | Reglas GeCode      |
| MotorLogico_Arquitectura-1.pdf | 29 KB  | Arquitectura motor |
| MPG.pdf                        | 2.1 MB | Manual completo    |

## 11. Métricas del Proyecto

### 11.1 Código Fuente

| Métrica         | Valor         |
|-----------------|---------------|
| Archivos Pascal | 19            |
| Archivos C      | 7             |
| Archivos H      | 1             |
| Total archivos  | 49            |
| SLOC estimado   | ~8000+ líneas |

### 11.2 Compilación

| Métrica            | Valor        |
|--------------------|--------------|
| Ejecutables        | 9            |
| Tamaño total       | 2.2 MB       |
| Tamaño obj/        | 1.2 MB       |
| Tiempo compilación | ~15 segundos |

### 11.3 Ejecutables

| Programa        | Tamaño |
|-----------------|--------|
| BwdConsistency  | 272 KB |
| CSPEval         | 264 KB |
| ForwardChain    | 268 KB |
| FunctionChecker | 221 KB |
| FwdConsistency  | 272 KB |
| JsonSink        | 225 KB |
| JsonSource      | 210 KB |
| JsonToGraph     | 256 KB |
| SyntaxChecker   | 261 KB |

### 11.4 Documentación

| Métrica            | Valor  |
|--------------------|--------|
| Archivos .md       | 6+     |
| Archivos .txt      | 19     |
| PDFs técnicos      | 7      |
| Tamaño total docs/ | 2.5 MB |

---

## 12. Ventajas de la Arquitectura Modular

### 12.1 Filosofía Unix

| Principio                     | Aplicación                                                  |
|-------------------------------|-------------------------------------------------------------|
| <b>Programas pequeños</b>     | 9 ejecutables standalone, cada uno < 300 KB                 |
| <b>Cada uno hace una cosa</b> | SyntaxChecker solo valida, JsonToGraph solo convierte, etc. |
| <b>Se pueden encadenar</b>    | Pipeline vía JSON (stdout -> stdin), componible             |

### 12.2 Beneficios

#### 1. Composición Flexible

- Pipeline se puede reorganizar sin recompilar
- Etapas se pueden omitir o reemplazar
- Fácil paralelización (GNU parallel)

#### 2. Debugging Sencillo

- Cada etapa se prueba independientemente
- Salida JSON intermedia inspeccionable
- Errores localizados por etapa

#### 3. Extensibilidad

- Agregar nueva etapa = nuevo programa
- Sin modificar código existente

- API estable (formato JSON)
4. **Portabilidad**
    - Sin dependencias complejas
    - Compilación limpia (Pascal + C estándar)
    - Scripts shell portables
  5. **Mantenibilidad**
    - Módulos pequeños y comprensibles
    - Cambios aislados por programa
    - Refactorización segura
- 

## 13. Flujo de Trabajo

### 13.1 Compilar el Proyecto

```
cd /home/rodo/parser
make clean
make
```

### 13.2 Ejecutar Pipeline Completo

```
./csp-pipeline.sh json/example.json
```

### 13.3 Ejecutar Etapas Individualmente

```
# 1. Validar
./SyntaxChecker json/example.json

# 2. Generar grafo
./JsonToGraph json/example.json > /tmp/graph.json

# 3. Verificar funciones
./FunctionChecker /tmp/graph.json --path ./lib:./obj

# 4. Propagar
./CSPEval /tmp/graph.json > /tmp/result.json

# 5. Inspeccionar resultado
cat /tmp/result.json | jq .
```

### 13.4 Ejecutar Tests Unitarios

```
cd test
./TestRoaring
./TestLabeledSet
./TestMiniMath
./TestMiniInterval
./TestMiniComplex
```

## 13.5 Agregar Nueva Función User-Defined

### 1. Escribir función en C:

```
// mylib/mifunc.c
#include <math.h>

void mifunc(double *inputs, int n_in, double *outputs, int n_out) {
    if (n_in >= 1 && n_out >= 1) {
        outputs[0] = sqrt(inputs[0]);
    }
}
```

### 2. Compilar a .o:

```
gcc -c -O2 -fPIC mylib/mifunc.c -o mylib/mifunc.o
```

### 3. Declarar en JSON:

```
{
  "functions": [
    {
      "name": "mifunc",
      "inputs": [{"name": "x", "type": "numeric", "domain": [0.0, 100.0]}],
      "outputs": [{"name": "r", "type": "numeric", "domain": [0.0, 10.0]}]
    }
  ]
}
```

### 4. Usar en expresión:

```
{
  "constraints": ["y = mifunc(x)"]
}
```

### 5. Ejecutar pipeline con path:

```
./FunctionChecker graph.json --path ./mylib:./lib:./obj
./CSPEval graph.json
```

---

## 14. Integración con GecodeInfo

### 14.1 Análisis del Espacio de Soluciones

Después de CSPEval, se puede analizar el espacio de soluciones con GecodeInfo:

```
# 1. Pipeline CSP
./csp-pipeline.sh json/example.json > /tmp/graph.json

# 2. Análisis con Gecode
cd /home/rodo/gecode
./GecodeInfo /tmp/graph.json
```

## Output de GecodeInfo:

```
{
  "total_solutions": 42,
  "analysis": [
    {"name": "x", "type": "numeric", "if_fixed": false, "values": [1.5, 2.3, 5.7]},
    {"name": "y", "type": "integer", "if_fixed": true, "value": 10}
  ]
}
```

## 14.2 Bug Crítico Resuelto (Interval Nodes)

**Problema:** Nodos CT\_INTERVAL en AST usan campos "lo"/"hi" no "left"/"right".

**Solución en UCSPJson.pas (GecodeInfo):**

```
// ParseNodeBlock - al final:
if Node.Left = -1 then begin
  P := FindKey(Blk, 'lo', 1);
  if P > 0 then Node.Left := ReadInt(Blk, P);
end;
if Node.Right = -1 then begin
  P := FindKey(Blk, 'hi', 1);
  if P > 0 then Node.Right := ReadInt(Blk, P);
end;
```

Sin esto, CT\_IN\_INTERVAL queda Lo=0, Hi=0 -> modelo FAILED.

---

## 15. Próximos Pasos

### 15.1 Funcionalidades Pendientes

- ☐ Completar documentación Markdown (01-04)
- ☐ Generar PDFs con pandoc
- ☐ Agregar tests de integración (pipeline completo)
- ☐ Implementar cache de resultados (CSPEval)
- ☐ Optimización de propagación (skip constraints sin cambios)
- ☐ Soporte para arrays multidimensionales

### 15.2 Mejoras de Infraestructura

- ☐ CI/CD script (compilación + tests automáticos)
- ☐ Cobertura de código (fpc -CR)
- ☐ Profiling de performance
- ☐ Benchmarks de propagación
- ☐ Script de empaquetado (.tar.gz)

### 15.3 Documentación

- ☐ Generar Doxygen para código C

- ☐ Diagramas de flujo (pipeline)
  - ☐ Ejemplos comentados (json/)
  - ☐ Tutorial paso a paso
- 

## 16. Conclusiones

### 16.1 Logros

- **[OK]** Arquitectura modular Unix (9 programas standalone)
- **[OK]** Pipeline CSP completo y funcional
- **[OK]** 4 tipos de variables soportados
- **[OK]** Grid bitmask 20-pt para variables numeric
- **[OK]** Aritmética de intervalos completa
- **[OK]** 8 reglas semánticas implementadas
- **[OK]** Verificación de funciones user-defined
- **[OK]** 4 motores de propagación diferentes
- **[OK]** Documentación estructurada y completa

### 16.2 Estado del Proyecto

- **Código:** Compilable, modular, sin warnings
- **Tests:** Unitarios completos para cada módulo
- **Documentación:** En progreso (README + docs/)
- **Pipeline:** Funcional de extremo a extremo
- **Integración:** GecodeInfo compatible

### 16.3 Filosofía de Diseño

“Unix se diseñó con 3 principios: Programas pequeños, Cada uno hace una cosa, Se pueden encadenar. Eso es pipeline.”

Este proyecto encarna esos principios: 9 programas pequeños (< 300 KB), cada uno con responsabilidad única, componibles vía JSON, extensibles sin modificar código existente.

---

## Fin del Documento de Estructura